MongoDB — Complete Notes

From Beginner to Advanced

CHAPTER 1 — Introduction

What is MongoDB? :-

MongoDB is a **NoSQL, document-oriented** database. Instead of storing data in rows and columns like a traditional relational database (MySQL, PostgreSQL), MongoDB stores data as **documents** — which look like JSON objects.

- The name **"Mongo"** comes from "huMONGOus" — meaning it can handle huge amounts of data.
- MongoDB is open-source and developed by **MongoDB Inc.**
- It is written in **C++**.
- MongoDB is a **schema-less** database — meaning you do NOT have to define a fixed structure before inserting data.

Ex:- Storing student data in MongoDB looks like this:

```json
{
  "name": "Ravi",
  "age": 21,
  "branch": "CSE",
  "subjects": ["Math", "OS", "DBMS"]
}
```

Why MongoDB? — Problems with SQL :-

In SQL databases, every row in a table must follow the same structure (same columns). This causes problems when:

- Different records need different fields (e.g., one product has "color", another has "size").
- You need to store nested or hierarchical data (e.g., address with city, state, pincode).

- You need to scale horizontally across multiple servers.

MongoDB solves all of this by storing each record as a flexible document.

SQL vs MongoDB — Terminology Comparison :-

SQL Term	MongoDB Term	Description
Database	Database	Same concept
Table	Collection	Group of documents
Row / Record	Document	A single data entry
Column	Field	A key in the document
Primary Key	`_id`	Unique identifier
JOIN	`$lookup`	Combine data from collections
Index	Index	Same concept
View	View	Same concept
Stored Procedure	Aggregation Pipeline	Data processing

Key Features of MongoDB :-

- **Document-Oriented** — Data stored as BSON documents (Binary JSON).
- **Schema-less** — No fixed structure needed. Each document can have different fields.
- **Scalable** — Supports horizontal scaling through **Sharding**.
- **High Performance** — Indexing, in-memory processing, fast reads/writes.
- **High Availability** — Through **Replica Sets** (copies of your data on multiple servers).
- **Rich Query Language** — Powerful querying, filtering, aggregation.
- **Supports Arrays & Nested Objects** — Natively, without joins.
- **Geospatial Support** — Can store and query location data.
- **Full-text Search** — Built-in text search capabilities.

What is BSON? :-

BSON stands for **Binary JSON**. MongoDB stores documents internally in BSON format.

- BSON extends JSON with additional data types like `Date`, `ObjectId`, `Binary`, `Int32`, `Int64`.
- BSON is faster for machines to encode/decode compared to plain JSON text.
- When you write queries in MongoDB shell, you write in JSON-like syntax. MongoDB converts it to BSON internally.

Ex:- JSON vs BSON data types:

JSON Type	BSON Extra Types
String	String
Number	Int32, Int64, Double, Decimal128
Boolean	Boolean
Array	Array
Object	Document
Null	Null
—	Date
—	ObjectId
—	Binary Data
—	Regular Expression
—	Timestamp

CHAPTER 2 — Installation on Linux (Ubuntu/Debian)

Step 1 — Import MongoDB GPG Key :-

The GPG key verifies the authenticity of MongoDB packages.

```bash
curl -fsSL https://www.mongodb.org/static/pgp/server-7.0.asc | \
   sudo gpg -o /usr/share/keyrings/mongodb-server-7.0.gpg \
   --dearmor
```

Step 2 — Add MongoDB Repository :-

```bash
echo "deb [ arch=amd64,arm64 signed-by=/usr/share/keyrings/mongodb-server-7.0.gpg ]
https://repo.mongodb.org/apt/ubuntu jammy/mongodb-org/7.0 multiverse" | \
sudo tee /etc/apt/sources.list.d/mongodb-org-7.0.list
```

> **Note:-** Replace `jammy` with your Ubuntu version codename.
>
> - Ubuntu 22.04 → `jammy`
> - Ubuntu 20.04 → `focal`

Step 3 — Update Package List :-

```bash
sudo apt-get update
```

Step 4 — Install MongoDB :-

```bash
sudo apt-get install -y mongodb-org
```

This installs:

- `mongod` — the MongoDB server (daemon)

- `mongos` — the MongoDB sharding router
- `mongosh` — the MongoDB shell (command-line client)
- `mongodump`, `mongorestore` — backup/restore tools

Step 5 — Start MongoDB Service :-

```bash
sudo systemctl start mongod
```

Step 6 — Enable MongoDB to Start on Boot :-

```bash
sudo systemctl enable mongod
```

Step 7 — Check MongoDB Status :-

```bash
sudo systemctl status mongod
```

You should see **Active: active (running)** in green.

Step 8 — Access MongoDB Shell :-

```bash
mongosh
```

You will see a prompt like:

```
Current Mongosh Log ID: ...
Connecting to: mongodb://127.0.0.1:27017/
test>
```

Other Useful Service Commands :-

```bash
sudo systemctl stop mongod       # Stop MongoDB
sudo systemctl restart mongod    # Restart MongoDB
sudo systemctl reload mongod     # Reload config without restart
```

MongoDB Configuration File :-

MongoDB's main config file is located at:

```
/etc/mongod.conf
```

Important settings inside:

```yaml
storage:
  dbPath: /var/lib/mongodb      # Where data files are stored

net:
  port: 27017                   # Default port
  bindIp: 127.0.0.1             # Listen on localhost only

systemLog:
  path: /var/log/mongodb/mongod.log
```

After changing the config, restart MongoDB:

```bash
sudo systemctl restart mongod
```

Uninstall MongoDB (if needed) :-

```bash
```

```
sudo systemctl stop mongod
sudo apt-get purge mongodb-org*
sudo rm -r /var/log/mongodb
sudo rm -r /var/lib/mongodb
```

CHAPTER 3 — MongoDB Shell (mongosh) Basics

---

mongosh — Command Line Tool :-

`mongosh` is the interactive JavaScript shell for MongoDB. You can run all database commands from here.

Useful Shell Commands :-

Command	Purpose
`show dbs`	List all databases
`use dbname`	Switch to (or create) a database
`db`	Show current database
`show collections`	List all collections in current DB
`exit` or `quit()`	Exit the shell
`cls`	Clear the screen
`help`	Show help menu
`db.stats()`	Show database statistics
`db.collection.stats()`	Show collection statistics

Creating / Switching a Database :-

```
js
```

```
use school
```

> **Note:-** MongoDB does NOT create the database immediately. It is created only when you insert the first document into a collection.

Deleting a Database :-

```js
use school
db.dropDatabase()
```

Creating a Collection :-

Method 1 — Implicit (auto-created on first insert):

```js
db.students.insertOne({ name: "Ravi" })
// 'students' collection is created automatically
```

Method 2 — Explicit:

```js
db.createCollection("students")
```

Creating a Collection with Options :-

```js
db.createCollection("logs", {
  capped: true,       // Fixed-size collection (old data deleted when full)
  size: 1048576,      // Max size in bytes (1 MB)
  max: 1000           // Max number of documents
})
```

Deleting a Collection :-

```js
db.students.drop()
```

CHAPTER 4 — CRUD Operations

CRUD stands for **Create, Read, Update, Delete** — the four basic operations on any database.

C — CREATE (Inserting Documents)

---

insertOne() :-

Inserts a single document into a collection.

```js
db.students.insertOne({
  name: "Ravi",
  age: 21,
  branch: "CSE",
  city: "Hyderabad"
})
```

Result:

```json
{
  "acknowledged": true,
  "insertedId": ObjectId("65f1a2b3c4d5e6f7a8b9c0d1")
}
```

> **Note:-** MongoDB automatically adds a unique `_id` field (ObjectId) to every document. You can also provide your own `_id`.

---

insertMany() :-

Inserts multiple documents at once. Pass an array of documents.

```js
db.students.insertMany([
  { name: "Priya", age: 20, branch: "ECE", city: "Mumbai" },
  { name: "Raman", age: 22, branch: "MECH", city: "Delhi" },
  { name: "Ankit", age: 21, branch: "CSE", city: "Pune" }
])
```

Result:

```json
{
  "acknowledged": true,
  "insertedIds": {
    "0": ObjectId("..."),
    "1": ObjectId("..."),
    "2": ObjectId("...")
  }
}
```

The _id Field :-

- Every document in MongoDB **must** have a unique `_id` field.
- If you don't provide it, MongoDB automatically generates an **ObjectId**.
- An **ObjectId** is a 12-byte unique identifier:
  - 4 bytes → timestamp (seconds since Unix epoch)
  - 5 bytes → random value (unique per machine & process)
  - 3 bytes → incrementing counter

```js
```

```
// Providing your own _id
db.students.insertOne({ _id: 1001, name: "Ravi" })

// Using ObjectId explicitly
db.students.insertOne({ _id: ObjectId(), name: "Priya" })
```

R — READ (Querying Documents)

---

find() :-

Returns all documents in a collection (like `SELECT * FROM table`).

```
js

db.students.find()
```

To display results in a readable format:

```
js

db.students.find().pretty()
```

> **Note:-** In newer versions of mongosh, `.pretty()` is applied automatically.

---

findOne() :-

Returns only the **first** document that matches the condition.

```
js

db.students.findOne({ name: "Ravi" })
```

---

find() with a Filter (WHERE clause equivalent) :-

```
js
```

```
// Find students where branch is CSE
db.students.find({ branch: "CSE" })

// Find students where age is 21
db.students.find({ age: 21 })

// Find students where city is Hyderabad AND branch is CSE
db.students.find({ city: "Hyderabad", branch: "CSE" })
```

Projection — Selecting Specific Fields :-
Projection controls which fields to include or exclude in results (like `SELECT col1, col2` in SQL).

```
js

// Syntax: db.collection.find(filter, projection)

// Include only name and branch (exclude _id)
db.students.find({}, { name: 1, branch: 1, _id: 0 })

// Exclude age and city
db.students.find({}, { age: 0, city: 0 })
```

> **Rule:-** You cannot mix include (1) and exclude (0) in the same projection, EXCEPT for `_id` which can always be explicitly excluded.

Comparison Query Operators :-

These are used inside `find()` to filter documents based on conditions.

Operator	Meaning	SQL Equivalent
`$eq`	Equal to	`=`
`$ne`	Not equal to	`!=`
`$gt`	Greater than	`>`
`$gte`	Greater than or equal	`>=`
`$lt`	Less than	`<`
`$lte`	Less than or equal	`<=`
`$in`	Value in array	`IN (...)`
`$nin`	Value NOT in array	`NOT IN (...)`

Syntax:-

```js
{ field: { $operator: value } }
```

Examples:-

```js
// age > 20
db.students.find({ age: { $gt: 20 } })

// age >= 21 AND age <= 25
db.students.find({ age: { $gte: 21, $lte: 25 } })

// branch is either CSE or ECE
db.students.find({ branch: { $in: ["CSE", "ECE"] } })

// branch is NOT MECH
db.students.find({ branch: { $ne: "MECH" } })

// age not in [20, 21]
db.students.find({ age: { $nin: [20, 21] } })
```

Logical Query Operators :-

Operator	Meaning	SQL Equivalent
`$and`	All conditions must be true	`AND`
`$or`	At least one condition true	`OR`
`$not`	Negate a condition	`NOT`
`$nor`	None of the conditions true	`NOR`

Examples:-

```
js
```

```
// age > 20 AND branch = "CSE"
db.students.find({
  $and: [
    { age: { $gt: 20 } },
    { branch: "CSE" }
  ]
})

// city = "Hyderabad" OR city = "Mumbai"
db.students.find({
  $or: [
    { city: "Hyderabad" },
    { city: "Mumbai" }
  ]
})

// NOT (branch = "MECH")
db.students.find({
  branch: { $not: { $eq: "MECH" } }
})

// Neither city is Delhi NOR branch is MECH
db.students.find({
  $nor: [
    { city: "Delhi" },
    { branch: "MECH" }
  ]
})
```

Element Query Operators :-

Operator	Meaning
`$exists`	Check if a field exists in the document
`$type`	Check if a field is of a certain BSON type

```
js
```

```
// Find documents where 'phone' field exists
db.students.find({ phone: { $exists: true } })

// Find documents where 'phone' field does NOT exist
db.students.find({ phone: { $exists: false } })

// Find where 'age' is of type 'int'
db.students.find({ age: { $type: "int" } })
```

Array Query Operators :-

Operator	Meaning
`$all`	Array contains all specified values
`$elemMatch`	At least one element matches all conditions
`$size`	Array has exactly n elements

js

```
// subjects array contains BOTH "Math" AND "OS"
db.students.find({ subjects: { $all: ["Math", "OS"] } })

// Array has exactly 3 elements
db.students.find({ subjects: { $size: 3 } })

// At least one score matches ALL conditions
db.students.find({
  scores: { $elemMatch: { $gte: 80, $lt: 90 } }
})
```

Querying Nested Documents :-

Use **dot notation** to query inside nested objects.

js

```
// Document structure:
// { name: "Ravi", address: { city: "Hyderabad", state: "Telangana" } }

// Query nested field
db.students.find({ "address.city": "Hyderabad" })

// Query nested field with condition
db.students.find({ "address.state": "Telangana" })
```

Cursor Methods :-

`find()` returns a **cursor** (a pointer to results). You can chain methods on it.

limit() :-

Restricts number of documents returned.

```
js

db.students.find().limit(5)   // Return only 5 documents
```

skip() :-

Skips the first N documents. Used for pagination.

```
js

db.students.find().skip(10)   // Skip first 10, return rest
```

Pagination Example :-

```
js

// Page 1: first 10 results
db.students.find().limit(10).skip(0)

// Page 2: next 10 results
db.students.find().limit(10).skip(10)

// Page 3
db.students.find().limit(10).skip(20)
```

sort() :-

Sorts results. `1` = ascending, `-1` = descending.

```js
// Sort by age ascending
db.students.find().sort({ age: 1 })

// Sort by age descending
db.students.find().sort({ age: -1 })

// Sort by branch ascending, then age descending
db.students.find().sort({ branch: 1, age: -1 })
```

count() / countDocuments() :-

Returns the number of matching documents.

```js
db.students.countDocuments()
db.students.countDocuments({ branch: "CSE" })
```

Chaining All Together :-

```js
db.students
  .find({ branch: "CSE" })
  .sort({ age: 1 })
  .skip(5)
  .limit(10)
```

U — UPDATE (Modifying Documents)

Update Operators :-

Before learning update commands, you need to know the update operators:

Operator	Meaning
`$set`	Set a field value (add or update)
`$unset`	Remove a field from the document
`$inc`	Increment a numeric field by a value
`$dec`	Decrement (use `$inc` with negative value)
`$mul`	Multiply a field by a value
`$rename`	Rename a field
`$min`	Update only if new value is less than current
`$max`	Update only if new value is greater than current
`$push`	Add an element to an array
`$pull`	Remove elements from an array
`$addToSet`	Add to array only if not already present
`$pop`	Remove first or last element of array

---

updateOne() :-

Updates the **first** document that matches the filter.

```
js
```

```
// Syntax
db.collection.updateOne(filter, update, options)

// Set age to 22 for first document where name is "Ravi"
db.students.updateOne(
  { name: "Ravi" },
  { $set: { age: 22 } }
)

// Add a new field 'email' to first matching document
db.students.updateOne(
  { name: "Ravi" },
  { $set: { email: "ravi@example.com" } }
)

// Increment marks by 5
db.students.updateOne(
  { name: "Ravi" },
  { $inc: { marks: 5 } }
)

// Remove the 'city' field
db.students.updateOne(
  { name: "Ravi" },
  { $unset: { city: "" } }
)
```

---

updateMany() :-

Updates **all** documents that match the filter.

```
js
```

```
// Set status to "active" for ALL CSE students
db.students.updateMany(
  { branch: "CSE" },
  { $set: { status: "active" } }
)

// Add 10 marks to all students where age > 20
db.students.updateMany(
  { age: { $gt: 20 } },
  { $inc: { marks: 10 } }
)

// Update ALL documents (empty filter = match all)
db.students.updateMany(
  {},
  { $set: { semester: 3 } }
)
```

---

replaceOne() :-

Replaces the **entire document** (except `_id`) with a new document.

```
db.students.replaceOne(
  { name: "Ravi" },
  { name: "Ravi Kumar", age: 23, branch: "IT" }
)
```

> **Warning:-** Unlike `$set`, `replaceOne()` removes ALL existing fields and replaces with new ones.

---

upsert Option :-

If `upsert: true` is set and no document matches the filter, MongoDB **inserts a new document** instead of doing nothing.

```
```

```
db.students.updateOne(
  { name: "Deepak" },
  { $set: { age: 24, branch: "ECE" } },
  { upsert: true }
)
// If "Deepak" doesn't exist, creates a new document
```

Updating Array Fields :-

```
js

// Push one subject to the array
db.students.updateOne(
  { name: "Ravi" },
  { $push: { subjects: "Networks" } }
)

// Push multiple subjects at once
db.students.updateOne(
  { name: "Ravi" },
  { $push: { subjects: { $each: ["Networks", "AI"] } } }
)

// Remove "Math" from subjects array
db.students.updateOne(
  { name: "Ravi" },
  { $pull: { subjects: "Math" } }
)

// Add only if not already present
db.students.updateOne(
  { name: "Ravi" },
  { $addToSet: { subjects: "DBMS" } }
)

// Remove last element of array
db.students.updateOne(
  { name: "Ravi" },
  { $pop: { subjects: 1 } }   // 1 = last, -1 = first
)
```

D — DELETE (Removing Documents)

---

deleteOne() :-

Deletes the **first** document that matches the filter.

```
js

db.students.deleteOne({ name: "Ravi" })

db.students.deleteOne({ age: { $lt: 18 } })
```

---

deleteMany() :-

Deletes **all** documents that match the filter.

```
js

// Delete all MECH students
db.students.deleteMany({ branch: "MECH" })

// Delete all documents (DANGEROUS — empties the collection)
db.students.deleteMany({})
```

---

findOneAndDelete() :-

Deletes the first matching document AND returns it.

```
js

const deleted = db.students.findOneAndDelete({ name: "Ravi" })
// Returns the deleted document
```

---

findOneAndUpdate() :-

Updates the first matching document AND returns it (before or after update).

```
js
```

```
db.students.findOneAndUpdate(
  { name: "Ravi" },
  { $set: { age: 25 } },
  { returnDocument: "after" }   // "before" or "after"
)
```

CHAPTER 5 — Indexes

What is an Index? :-

An index is a special data structure that stores a small portion of the collection's data in an easy-to-traverse form. Without an index, MongoDB must perform a **collection scan** (checking every document) — which is very slow for large collections.

With an index, MongoDB can jump directly to the matching documents.

Think of it like a book index — instead of reading every page, you look up the term in the index and go to that page directly.

Types of Indexes :-

1. Single Field Index :-

Index on one field.

```
js

// Create ascending index on 'name' field
db.students.createIndex({ name: 1 })

// Create descending index
db.students.createIndex({ age: -1 })
```

2. Compound Index :-

Index on multiple fields together.

```
js
```

```
db.students.createIndex({ branch: 1, age: -1 })
// Useful for queries like: find({ branch: "CSE" }).sort({ age: -1 })
```

3. Unique Index :-

Ensures all values in the field are unique.

```
js

db.students.createIndex({ email: 1 }, { unique: true })
```

4. Text Index :-

Enables full-text search on string fields.

```
js

db.articles.createIndex({ content: "text" })
db.articles.find({ $text: { $search: "mongodb database" } })
```

5. Sparse Index :-

Only indexes documents that HAVE the indexed field (skips documents where the field is missing).

```
js

db.students.createIndex({ phone: 1 }, { sparse: true })
```

6. TTL Index (Time-To-Live) :-

Automatically deletes documents after a specified time. Useful for sessions, logs, caches.

```
js

// Delete documents 3600 seconds (1 hour) after 'createdAt' date
db.sessions.createIndex({ createdAt: 1 }, { expireAfterSeconds: 3600 })
```

7. Wildcard Index :-

Indexes all fields (or all fields within a nested object).

```
js

db.products.createIndex({ "$**": 1 })
```

Index Management :-

```js
// List all indexes on a collection
db.students.getIndexes()

// Drop a specific index
db.students.dropIndex({ name: 1 })
// or by index name
db.students.dropIndex("name_1")

// Drop ALL indexes (keeps _id index)
db.students.dropIndexes()
```

EXPLAIN — Query Analysis :-

Use `explain()` to understand how MongoDB executes a query and whether it's using indexes.

```js
db.students.find({ name: "Ravi" }).explain("executionStats")
```

Look for:

- `COLLSCAN` → Collection scan (bad, no index used)
- `IXSCAN` → Index scan (good, index used)
- `totalDocsExamined` → Fewer = better

CHAPTER 6 — Aggregation Pipeline

What is the Aggregation Pipeline? :-

The Aggregation Pipeline is MongoDB's way to process and transform data — like `GROUP BY`, `JOIN`, `HAVING`, `SELECT` with computed fields in SQL.

It works like a **pipe** — data flows through a series of **stages**, and each stage transforms the data.

```
Collection → Stage 1 → Stage 2 → Stage 3 → ... → Result
```

Syntax:-

```js
db.collection.aggregate([
  { $stage1: { ... } },
  { $stage2: { ... } },
  { $stage3: { ... } }
])
```

Aggregation Stages :-

$match — Filter Documents :-

Like `WHERE` in SQL. Filters documents that pass through to the next stage.

```js
db.students.aggregate([
  { $match: { branch: "CSE" } }
])

// With condition
db.students.aggregate([
  { $match: { age: { $gte: 20 } } }
])
```

$group — Group Documents :-

Like `GROUP BY` in SQL. Groups documents by a field and applies aggregate functions.

```js
```

```
// Count students per branch
db.students.aggregate([
  {
    $group: {
      _id: "$branch",         // Group by branch
      total: { $sum: 1 }    // Count documents
    }
  }
])

// Average age per branch
db.students.aggregate([
  {
    $group: {
      _id: "$branch",
      avgAge: { $avg: "$age" },
      maxAge: { $max: "$age" },
      minAge: { $min: "$age" },
      totalStudents: { $sum: 1 }
    }
  }
])

// Sum of salary per city
db.employees.aggregate([
  {
    $group: {
      _id: "$city",
      totalSalary: { $sum: "$salary" }
    }
  }
])
```

$group Accumulator Operators :-

Operator	Meaning
`$sum`	Sum of values (use 1 to count)
`$avg`	Average of values
`$min`	Minimum value

Operator	Meaning
`$max`	Maximum value
`$first`	First value in the group
`$last`	Last value in the group
`$push`	Collect all values into an array
`$addToSet`	Collect unique values into an array

$project — Shape the Output :-

Like `SELECT col1, col2` in SQL. Controls which fields appear in output and can create computed fields.

```
js
```

```
// Include only name and branch
db.students.aggregate([
  { $project: { name: 1, branch: 1, _id: 0 } }
])

// Create a computed field 'fullName'
db.students.aggregate([
  {
    $project: {
      fullName: { $concat: ["$firstName", " ", "$lastName"] },
      age: 1,
      _id: 0
    }
  }
])

// Convert age to string
db.students.aggregate([
  {
    $project: {
      name: 1,
      ageStr: { $toString: "$age" }
    }
  }
])
```

$sort — Sort Documents :-

Like `ORDER BY` in SQL.

```
db.students.aggregate([
  { $sort: { age: -1 } }    // Sort by age descending
])

db.students.aggregate([
  { $sort: { branch: 1, age: -1 } }  // Multi-field sort
])
```

$limit and $skip :-

Like `LIMIT` and `OFFSET` in SQL.

```js
db.students.aggregate([
  { $limit: 5 }     // Return first 5
])

db.students.aggregate([
  { $skip: 10 },    // Skip first 10
  { $limit: 5 }     // Then take next 5
])
```

$unwind — Deconstruct Arrays :-

Breaks an array field into separate documents — one document per array element.

```js
// Document: { name: "Ravi", subjects: ["Math", "OS", "DBMS"] }

db.students.aggregate([
  { $unwind: "$subjects" }
])

// Result:
// { name: "Ravi", subjects: "Math" }
// { name: "Ravi", subjects: "OS" }
// { name: "Ravi", subjects: "DBMS" }
```

$lookup — JOIN Collections :-

Like `JOIN` in SQL. Combines documents from two collections.

```js
```

```
// Orders collection: { _id: 1, product_id: 101, quantity: 2 }
// Products collection: { _id: 101, name: "Laptop", price: 80000 }

db.orders.aggregate([
  {
    $lookup: {
      from: "products",          // Collection to join
      localField: "product_id",  // Field in orders
      foreignField: "_id",       // Field in products
      as: "productDetails"       // Name for the joined array
    }
  }
])

// Result:
// {
//   _id: 1, product_id: 101, quantity: 2,
//   productDetails: [{ _id: 101, name: "Laptop", price: 80000 }]
// }
```

$addFields — Add New Fields :-

Adds new fields to documents without removing existing ones.

```
js

db.students.aggregate([
  {
    $addFields: {
      passFail: { $cond: { if: { $gte: ["$marks", 40] }, then: "Pass", else: "Fail"
      gradeYear: 2024
    }
  }
])
```

$count — Count Documents :-

Counts the number of documents passing through the pipeline.

```
js
```

```
db.students.aggregate([
  { $match: { branch: "CSE" } },
  { $count: "total_cse_students" }
])
// Result: { total_cse_students: 45 }
```

$out — Save Result to a Collection :-

Writes the pipeline result into a new or existing collection.

```
js

db.students.aggregate([
  { $match: { branch: "CSE" } },
  { $out: "cse_students" }   // Saves results to 'cse_students' collection
])
```

Full Aggregation Pipeline Example :-

Find the top 3 branches by average marks, for students above age 18:

```
js

db.students.aggregate([
  { $match: { age: { $gt: 18 } } },          // Filter
  { $group: {                                  // Group by branch
      _id: "$branch",
      avgMarks: { $avg: "$marks" },
      count: { $sum: 1 }
  }},
  { $sort: { avgMarks: -1 } },               // Sort by avgMarks descending
  { $limit: 3 },                             // Take top 3
  { $project: {                              // Shape output
      branch: "$_id",
      avgMarks: 1,
      count: 1,
      _id: 0
  }}
])
```

Aggregation Expressions :-

Arithmetic Expressions :-

```js
{ $add: ["$price", "$tax"] }         // price + tax
{ $subtract: ["$total", "$discount"] } // total - discount
{ $multiply: ["$price", "$qty"] }    // price * qty
{ $divide: ["$total", "$count"] }    // total / count
{ $mod: ["$age", 2] }               // age % 2
{ $abs: "$temperature" }             // absolute value
{ $ceil: "$rating" }                 // round up
{ $floor: "$rating" }                // round down
{ $round: ["$rating", 2] }           // round to 2 decimals
{ $sqrt: "$area" }                   // square root
{ $pow: ["$base", "$exp"] }          // base^exp
```

String Expressions :-

```js
{ $concat: ["$firstName", " ", "$lastName"] }  // Concatenate
{ $toUpper: "$name" }                          // UPPERCASE
{ $toLower: "$name" }                          // lowercase
{ $trim: { input: "$name" } }                  // Remove spaces
{ $substr: ["$name", 0, 5] }                   // Substring from pos 0, length 5
{ $strLenCP: "$name" }                         // Length of string
{ $split: ["$fullName", " "] }                 // Split string by delimiter
{ $indexOfCP: ["$email", "@"] }                // Find position of "@"
```

Conditional Expressions :-

```js
```

```
// $cond — if / then / else
{ $cond: { if: { $gte: ["$marks", 40] }, then: "Pass", else: "Fail" } }

// $ifNull — return default if field is null
{ $ifNull: ["$phone", "Not Provided"] }

// $switch — like switch/case
{
  $switch: {
    branches: [
      { case: { $gte: ["$marks", 90] }, then: "A" },
      { case: { $gte: ["$marks", 75] }, then: "B" },
      { case: { $gte: ["$marks", 60] }, then: "C" }
    ],
    default: "F"
  }
}
```

Date Expressions :-

```
js

{ $year: "$createdAt" }           // Get year
{ $month: "$createdAt" }          // Get month (1-12)
{ $dayOfMonth: "$createdAt" }     // Get day (1-31)
{ $hour: "$createdAt" }           // Get hour
{ $minute: "$createdAt" }         // Get minute
{ $dateToString: {                 // Format date
    format: "%Y-%m-%d",
    date: "$createdAt"
}}
```

CHAPTER 7 — Schema Design & Data Modeling

Embedding vs Referencing :-

In MongoDB, when designing your data model, you have two main strategies:

---

1. Embedding (Denormalization) :-

Store related data **inside** the same document as a nested object or array.

```js
// Embedded model — student with addresses embedded
{
  _id: 1,
  name: "Ravi",
  addresses: [
    { type: "home", city: "Hyderabad", state: "Telangana" },
    { type: "college", city: "Pune", state: "Maharashtra" }
  ]
}
```

When to use Embedding:-

- The nested data is always accessed together with the parent.
- One-to-few relationships (e.g., a person and their phone numbers).
- Data doesn't grow unboundedly.

Pros:- Faster reads (no joins), simpler queries, atomic operations.

Cons:- Document size limit (16MB), duplication if shared across documents.

---

2. Referencing (Normalization) :-

Store related data in **separate collections** and use `_id` references (like foreign keys).

```js
// Students collection
{ _id: 1, name: "Ravi", course_id: ObjectId("abc123") }

// Courses collection
{ _id: ObjectId("abc123"), name: "B.Tech CSE", duration: "4 years" }
```

When to use Referencing:-

- One-to-many or many-to-many relationships.
- Data is shared across multiple documents.
- The embedded data would grow very large.

Pros:- No data duplication, flexible, handles large datasets.

Cons:- Requires multiple queries or `$lookup` to get related data.

Document Size Limit :-

MongoDB has a **16 MB maximum document size** limit. If your document might grow large (e.g., storing all posts of a user), use referencing instead of embedding.

CHAPTER 8 — Transactions

What are Transactions? :-

A transaction is a group of operations that either **all succeed** or **all fail together** (atomicity). This is critical for operations like banking — where money must be deducted from one account AND added to another simultaneously.

MongoDB supports **ACID transactions** from version 4.0 for replica sets and from version 4.2 for sharded clusters.

Single Document Atomicity :-

MongoDB operations on a **single document** are always atomic — even without explicit transactions. This is why embedding related data is powerful.

Multi-Document Transactions :-

For operations spanning multiple documents or collections, use explicit transactions.

```
js
```

```
// Start a session
const session = db.getMongo().startSession()

session.startTransaction()

try {
  // Deduct from account A
  db.accounts.updateOne(
    { _id: "A" },
    { $inc: { balance: -500 } },
    { session }
  )

  // Add to account B
  db.accounts.updateOne(
    { _id: "B" },
    { $inc: { balance: 500 } },
    { session }
  )

  // Commit if both succeed
  session.commitTransaction()
  print("Transaction committed successfully")

} catch (error) {
  // Rollback if anything fails
  session.abortTransaction()
  print("Transaction aborted: " + error)

} finally {
  session.endSession()
}
```

CHAPTER 9 — Replica Sets

What is a Replica Set? :-

A **Replica Set** is a group of MongoDB servers (nodes) that maintain the same copy of data. This provides:

- **High Availability** — If the primary server fails, a secondary automatically takes over.
- **Data Redundancy** — Multiple copies of your data on different machines.
- **Read Scaling** — You can read from secondary nodes to reduce load on primary.

Replica Set Members :-

Role	Description
Primary	The main node. Receives ALL write operations.
Secondary	Copies of the primary. Can serve reads.
Arbiter	Votes in elections but holds NO data. Used to break ties.

How Replica Sets Work :-

1. All writes go to the **Primary**.
2. The Primary records changes in an **Oplog** (Operations Log).
3. **Secondaries** replicate the Oplog and apply the same operations.
4. If Primary goes down, **election** happens — secondaries vote and one becomes the new Primary.

Setting up a Replica Set (basic) :-

bash

```bash
# Start 3 MongoDB instances on different ports
mongod --replSet "rs0" --port 27017 --dbpath /data/rs0
mongod --replSet "rs0" --port 27018 --dbpath /data/rs1
mongod --replSet "rs0" --port 27019 --dbpath /data/rs2
```

js

```
// Initialize replica set in mongosh
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "localhost:27017" },
    { _id: 1, host: "localhost:27018" },
    { _id: 2, host: "localhost:27019" }
  ]
})

// Check replica set status
rs.status()

// Check which node is primary
rs.isMaster()
```

CHAPTER 10 — Sharding

What is Sharding? :-

Sharding is MongoDB's method for **horizontal scaling** — distributing data across multiple machines (shards). When your dataset grows too large for a single machine, sharding splits the data.

How Sharding Works :-

- Data is divided into **chunks** based on a **shard key**.
- Each chunk is stored on a different **shard** (server).
- A **mongos** router directs queries to the correct shard(s).
- **Config Servers** store metadata about which chunks are on which shard.

```
Client → mongos (Router) → Config Servers (Metadata)
                         → Shard 1 (some data)
                         → Shard 2 (some data)
                         → Shard 3 (some data)
```

Shard Key :-

A **shard key** is the field (or fields) MongoDB uses to distribute data across shards.

```js
// Enable sharding for a database
sh.enableSharding("school")

// Create an index on the shard key field first
db.students.createIndex({ _id: "hashed" })

// Shard the collection
sh.shardCollection("school.students", { _id: "hashed" })
```

Types of Sharding :-

Type	Description
Range-based	Consecutive values go to the same shard. Good for range queries.
Hash-based	Hash of shard key distributes evenly. Good for uniform distribution.
Zone-based	You define which ranges go to which shard (e.g., by region).

CHAPTER 11 — Security

Enabling Authentication :-

By default, MongoDB has no authentication. For production, always enable it.

Step 1 — Create admin user (while auth is disabled):-

```js
```

```
use admin
db.createUser({
  user: "adminUser",
  pwd: "StrongPassword123",
  roles: [{ role: "userAdminAnyDatabase", db: "admin" }]
})
```

Step 2 — Enable auth in config file:-

yaml

```
# /etc/mongod.conf
security:
  authorization: enabled
```

Step 3 — Restart MongoDB:-

bash

```
sudo systemctl restart mongod
```

Step 4 — Connect with authentication:-

bash

```
mongosh -u adminUser -p StrongPassword123 --authenticationDatabase admin
```

Built-in Roles :-

Role	Access Level
`read`	Read documents in specific DB
`readWrite`	Read + Write in specific DB
`dbAdmin`	Database administration tasks
`userAdmin`	Create and manage users
`clusterAdmin`	Manage the whole cluster
`readAnyDatabase`	Read any database

Role	Access Level
`readWriteAnyDatabase`	Read + Write any database
`userAdminAnyDatabase`	Manage users in any database
`dbAdminAnyDatabase`	Admin any database
`root`	Superuser — all privileges

Creating Users with Specific Roles :-

```js
use school

db.createUser({
  user: "teacher",
  pwd: "teacher123",
  roles: [
    { role: "readWrite", db: "school" }
  ]
})

// Read-only user
db.createUser({
  user: "viewer",
  pwd: "view123",
  roles: [
    { role: "read", db: "school" }
  ]
})

// Drop a user
db.dropUser("viewer")
```

CHAPTER 12 — Backup & Restore

mongodump — Backup :-

`mongodump` exports data from MongoDB as BSON files.

```bash
# Backup entire MongoDB instance
mongodump --out /backup/mongodb/

# Backup specific database
mongodump --db school --out /backup/

# Backup specific collection
mongodump --db school --collection students --out /backup/

# With authentication
mongodump -u adminUser -p password --authenticationDatabase admin --out /backup/
```

---

mongorestore — Restore :-

`mongorestore` imports BSON backup files back into MongoDB.

```bash
# Restore entire backup
mongorestore /backup/mongodb/

# Restore specific database
mongorestore --db school /backup/school/

# Drop existing data before restoring
mongorestore --drop /backup/mongodb/
```

---

mongoexport / mongoimport — JSON/CSV :-

```bash
```

```
# Export collection to JSON
mongoexport --db school --collection students --out students.json

# Export collection to CSV
mongoexport --db school --collection students --type csv \
  --fields name,age,branch --out students.csv

# Import from JSON
mongoimport --db school --collection students --file students.json

# Import from CSV (with header row)
mongoimport --db school --collection students --type csv \
  --headerline --file students.csv
```

CHAPTER 13 — Advanced Topics

GridFS — Storing Large Files :-

MongoDB documents are limited to 16 MB. For larger files (images, videos, audio), use **GridFS** — it splits files into chunks and stores them across two collections:

- `fs.files` — File metadata
- `fs.chunks` — File chunks (255 KB each by default)

bash

```
# Store a file
mongofiles -d school put profile_photo.jpg

# List files
mongofiles -d school list

# Get (download) a file
mongofiles -d school get profile_photo.jpg

# Delete a file
mongofiles -d school delete profile_photo.jpg
```

Change Streams :-

Change streams allow you to **watch for real-time changes** in a collection or database. Used for:

- Real-time notifications
- Data synchronization
- Event-driven architectures

```js
// Watch all changes on students collection
const changeStream = db.students.watch()

// Process each change event
changeStream.forEach(change => {
  print(JSON.stringify(change))
})

// Watch only insert operations
const stream = db.students.watch([
  { $match: { operationType: "insert" } }
])
```

Change event includes:

- `operationType` — "insert", "update", "delete", "replace"
- `fullDocument` — The affected document
- `documentKey` — The `_id` of the affected document
- `updateDescription` — What changed (for updates)

Atlas Search (Full-Text Search) :-

MongoDB Atlas provides powerful full-text search using Apache Lucene.

For self-hosted, use the built-in text index:

```js
```

```js
// Create text index on multiple fields
db.articles.createIndex({ title: "text", content: "text" })

// Search for articles containing "mongodb" and "database"
db.articles.find({ $text: { $search: "mongodb database" } })

// Exclude a word
db.articles.find({ $text: { $search: "mongodb -SQL" } })

// Search for exact phrase
db.articles.find({ $text: { $search: "\"NoSQL database\"" } })

// Sort by text score (relevance)
db.articles.find(
  { $text: { $search: "mongodb" } },
  { score: { $meta: "textScore" } }
).sort({ score: { $meta: "textScore" } })
```

Geospatial Queries :-

MongoDB supports storing and querying geographic location data.

```
js
```

```
// Store location as GeoJSON point
db.places.insertOne({
  name: "Charminar",
  location: {
    type: "Point",
    coordinates: [78.4747, 17.3616]  // [longitude, latitude]
  }
})

// Create 2dsphere index for GeoJSON
db.places.createIndex({ location: "2dsphere" })

// Find places within 5 km of a point
db.places.find({
  location: {
    $near: {
      $geometry: { type: "Point", coordinates: [78.4747, 17.3616] },
      $maxDistance: 5000  // in meters
    }
  }
})

// Find places within a polygon
db.places.find({
  location: {
    $geoWithin: {
      $geometry: {
        type: "Polygon",
        coordinates: [[[78.4, 17.3], [78.5, 17.3], [78.5, 17.4], [78.4, 17.4], [78.4
      }
    }
  }
})
```

Capped Collections :-

A **capped collection** is a fixed-size collection. When it fills up, the oldest documents are automatically deleted to make room for new ones. Perfect for logs and event streams.

```
js
```

```
// Create a capped collection (max 1 MB, max 1000 docs)
db.createCollection("logs", {
  capped: true,
  size: 1048576,  // bytes
  max: 1000
})

// Check if a collection is capped
db.logs.isCapped()

// Convert existing collection to capped
db.runCommand({ convertToCapped: "logs", size: 1048576 })
```

Validation Rules (Schema Validation) :-

Even though MongoDB is schema-less, you can enforce validation rules on a collection.

```
js
```

```
db.createCollection("students", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "age", "branch"],
      properties: {
        name: {
          bsonType: "string",
          description: "Name must be a string and is required"
        },
        age: {
          bsonType: "int",
          minimum: 16,
          maximum: 35,
          description: "Age must be an integer between 16 and 35"
        },
        branch: {
          bsonType: "string",
          enum: ["CSE", "ECE", "MECH", "CIVIL", "IT"],
          description: "Branch must be one of the listed options"
        },
        email: {
          bsonType: "string",
          pattern: "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}$",
          description: "Must be a valid email address"
        }
      }
    }
  },
  validationAction: "error"   // "error" rejects, "warn" just logs
})
```

CHAPTER 14 — Performance & Optimization

Performance Best Practices :-

1. Always index fields used in queries:-

```
js
```

```
// Index fields used in find(), sort(), and aggregate $match
db.students.createIndex({ branch: 1 })
db.students.createIndex({ city: 1, age: -1 })
```

2. Use projection to fetch only needed fields:-

```
js

// BAD — fetches entire document
db.students.find({ branch: "CSE" })

// GOOD — fetches only needed fields
db.students.find({ branch: "CSE" }, { name: 1, marks: 1, _id: 0 })
```

3. Use $match early in aggregation pipelines:-

```
js

// BAD — $group processes all documents then filters
[{ $group: {...} }, { $match: { total: { $gt: 100 } } }]

// GOOD — $match first reduces data before $group
[{ $match: { age: { $gt: 18 } } }, { $group: {...} }]
```

4. Avoid large document scans:-

Use `explain("executionStats")` to check if indexes are being used.

5. Use connection pooling in your application to avoid creating a new connection for every request.

6. Limit document size — Keep documents under 1 MB for best performance.

Quick Reference — Common Commands :-

```
js
```

```
// DATABASE
show dbs
use mydb
db.dropDatabase()

// COLLECTION
show collections
db.createCollection("name")
db.collection.drop()
db.collection.stats()
db.collection.countDocuments()

// INSERT
db.col.insertOne({})
db.col.insertMany([{}, {}])

// READ
db.col.find()
db.col.find({filter})
db.col.find({filter}, {projection})
db.col.findOne({filter})
db.col.find().sort().skip().limit()

// UPDATE
db.col.updateOne({filter}, {$set: {}})
db.col.updateMany({filter}, {$set: {}})
db.col.replaceOne({filter}, {newDoc})

// DELETE
db.col.deleteOne({filter})
db.col.deleteMany({filter})

// INDEXES
db.col.createIndex({field: 1})
db.col.getIndexes()
db.col.dropIndex({field: 1})

// AGGREGATION
db.col.aggregate([pipeline stages])
```

MongoDB vs SQL — Quick Comparison :-

Feature	MongoDB	SQL (PostgreSQL/MySQL)
Data Format	BSON Documents	Tables (Rows & Columns)
Schema	Flexible (schema-less)	Fixed (schema required)
Joins	`$lookup` (complex)	JOIN (simple, powerful)
Transactions	Supported (v4.0+)	Native, fully supported
Scaling	Horizontal (Sharding)	Vertical (mainly)
Best For	Flexible, hierarchical data	Structured, relational data
Query Language	MongoDB Query Language (MQL)	SQL
Nested Data	Native support	Requires JSON/JSONB columns
Arrays	Native support	Requires ARRAY type
Full-text Search	Built-in (basic)	Built-in (advanced)

End of MongoDB Complete Notes